

COMP 2404 -- Tutorial #2

Basic C++ Classes

1. Learning Outcomes

After this tutorial, you will be able to:

- create a simple C++ class with data and behaviour
- implement a class constructor and other member functions

2. Required Background

To begin this tutorial, you must understand the following concepts from the course material:

- section 1.2: function design
- section 1.3: basic classes in C++, separating header and source files, constructors

If you are not familiar with this material, you **must** review it on your own before you can start this tutorial.

3. Tutorial Instructions

THE INSTRUCTIONS MUST BE COMPLETED IN ORDER.

1. You will begin with the base code provided in the `T02.tar` file, posted in *Brightspace*. Look through the content of each file to make sure that you understand what it does.
2. Implement the member functions in the `Book` class. This includes the following:
 - (1) a default constructor that takes four parameters, and initializes the corresponding data members to those values
 - (2) a getter member function for each data member
 - (3) a `set(int i, string t, string a, int y)` member function that sets the `Book` object's data members to the given parameters
 - (4) a `print()` function that prints out all the data members of the book.
3. Create a new `Library` class. You will need both a header file and a source file for this class. The class will contain two data members:
 - (1) a primitive array of `Book` objects
 - (2) the current number of books in the arraySince the book array functionality is moving from the `main.cc` file, you will also move the constant array size definition (`MAX_ARR_SIZE`) into the `Library` header file.
4. Implement the following member functions for the `Library` class:
 - (1) a constructor that initializes the data member(s) that require initialization; think about what these might be
 - (2) an `addBook(Book&)` function that adds the given book parameter to the back of the book array
 - (a) terminology: the *back* of a collection is its end; the *front* of a collection is its beginning
 - (b) the back of an array is considered to be one index past the index where the most recent book was added; so the first book is added to index zero, the next one is added to index 1, and so on

- (c) the new book must be added to the array by setting the data members of the destination book to the values of the data members in the source book
 - i. the *destination* book is the one at the correct index in the array
 - ii. the *source* book is found in the parameter
 - iii. you must use an existing function to set the destination book's data members
 - iv. **do not use the assignment operator to copy entire objects**
 - (d) remember to update the current number of books in the array
 - (3) a `print()` function that prints out all the books in the array to the screen by calling a function that you implemented in a previous step
5. Update the `main()` function as follows:
- (1) declare a `Library` object in local scope
 - (2) if the user chooses to add a book, create a temporary `Book` object to be added to the library, using the user-provided data
 - (3) add the new book to the library, using a function implemented in a previous step
 - (4) print the content of the library after the user exits the menu
6. Update the Makefile so that the new `Library` class gets compiled and linked into the executable, as we saw in the course material on Makefiles.
7. Look at the content of the `in.txt` file. Using Linux shell I/O redirection, connect the program executable's standard input to `in.txt` by typing on the command line: `t02 < in.txt`
8. Build and run the program. Check that the book information is correct when the library is printed out at the end of the program.
9. Create a readme file (`README`) that includes:
- (1) a preamble (program author, purpose, list of source files)
 - (2) the exact compilation command
 - (3) the launching and operating instructions
10. Create a tar file called `Tutorial02.tar` that contains your source and header files, your data file, your Makefile, and your `README` file. Make sure that there are no directories, no hidden directories, and no additional files in the tar file.
11. Upload and submit your `Tutorial02.tar` file to the Tutorial #2 assignment in *Brightspace*.

4. Supplementary Exercises (for additional practice only)

1. Implement a `bool delBook(int id)` member function in the `Library` class. This function removes from the library's book array the book with the given id. It returns true if the book was found and deleted, and false otherwise.
(1) you must "move" the remaining books one position towards the front of the array, in order to close the gap left by the deleted book; do not leave gaps in the array
2. Add a new option to the main menu that allows the user to select the "Delete book" option. The user must then be prompted to enter a book id, and the corresponding book must be deleted from the library. The `main()` function must print out a detailed error message if the deletion fails.
3. Implement a `bool findBook(int id, Book& b)` member function in the `Library` class. This function finds the book with the given id in the book array, and "returns" this book in the `b` parameter.
4. Add a new option to the main menu that allows the user to select the "Print book" option. The user must then be prompted to enter a book id, the corresponding book must be retrieved from the library, and that book's information must be printed to the screen using an existing function. The `main()` function must print out a detailed error message if the book is not found.
5. Build and run the program to test the new features. Check that the book information is correct when the library is printed out at the end of the program.
6. Save your work to a permanent location, like a memory stick or a shared folder.